Checkout Revamp Integration Guide

Version 6.1

July 29, 2020

Table of Contents

1. Introduction 4
2. Easypaisa Hosted Checkout 5
3. Integrating React Component 6
   - Step 1: Input the required information 6
   - Step 2: Paste the Easypaisa script inside web-site 7
   - Step 3: Call the Easypaisa script 8
   - Step 4: Create an iframe 9
   - Step 5: Sample String: 9
4. Encryption Algorithm 10
   - Step 1 10
   - Create a string in following format: 10
   - sampleString = amount=250&emailAddress=test@test.com&expiryDate= 10
   - If optional fields are not part of the request than they should also not be part of this string. 10
   - Step 2 10
   - Use AES/ECB/PKCS5Padding algorithm to encrypt with the key and string produced in the previous step 10
5. Transaction Status on Postback URL 12
6. RSA Encryption (Optional Pre Requisite) 14
   - Public/Private Key Exchange 14
   - Signature Generation 15
7. IPN Integration 17
   - Payments Flow 17
   - How to Integrate IPN Service 17
8. Errors in Payment Methods 19
   - MA 19
   - OTC 19
   - DD 20

CC 20

1. Introduction

Easypay Online Payment System (OPS) is an electronic payment solution that enables internet users to make financial transactions online. It seamlessly integrates with e-commerce web-sites, shopping carts or mobile apps and allows the owners of these applications to collect payments from their customers. Easypay allows partner businesses to offer their customers a variety of payment methods to pay for their transactions, which include Easypaisa Mobile Accounts, Credit/Debit Card payments, Internet Banking and others.

2. Easypaisa Hosted Checkout

Easypaisa Hosted Checkout is a lightweight, javascript component that can be embedded inside any e-commerce website with few simple steps of integration, and once integrated, allows web-site owners/ online merchants, to collect payments from their customers/viewers for their purchases, through a variety of payment methods offered by Easypaisa Payment Gateway.

The Easypaisa Hosted Checkout is a javascript function that can be called from inside the merchant website when a payment is required. The Checkout component renders itself inside an iframe within the merchant web-site, allowing the customers to interact with it to pay for their transactions, without re-directing out of the e-commerce site. The image below, shows what the Easypaisa Native Checkout utility looks like inside a merchant web page.

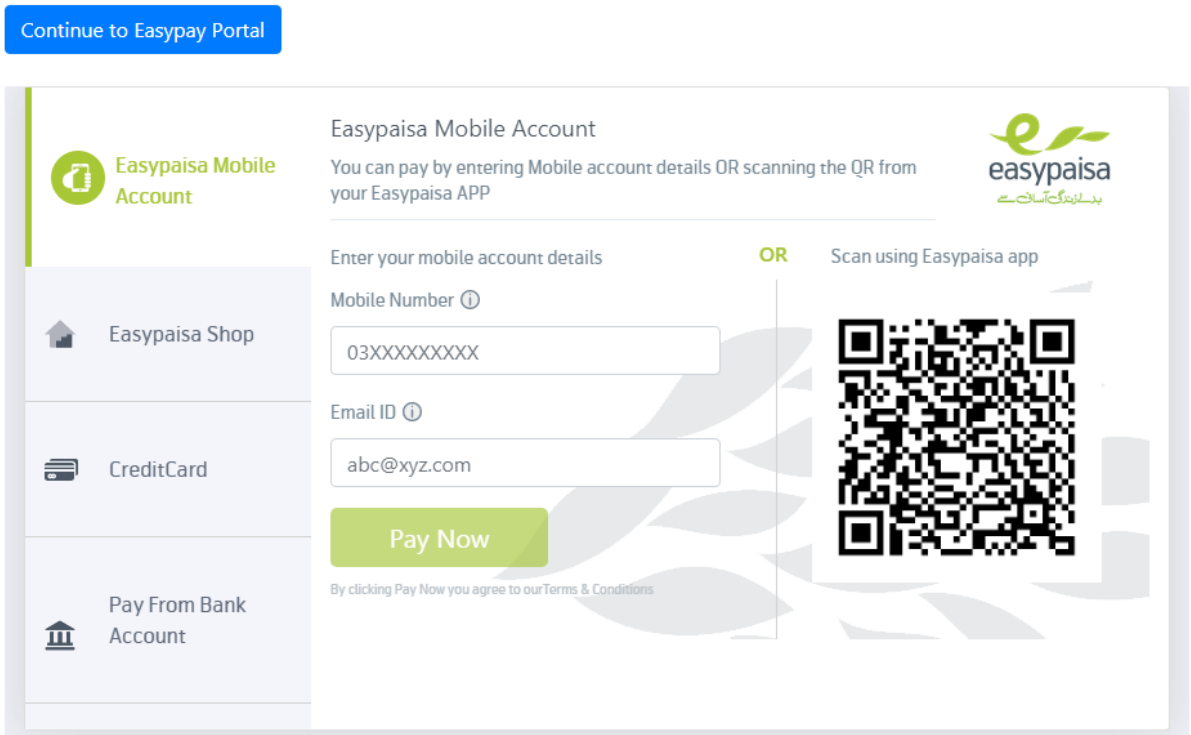


Figure 1 - Checkout component on loading

3. Integrating React Component

To integrate Easypaisa Hosted Checkout in an online webstore, website developers will need to include a javascript code snippet inside the website code. Secondly, they will need to ensure the availability of certain inputs in their website, and then call the inserted code snippet when a payment is needed. When the Input fields are received by the web-site, the Javascript function will read the input data and load the Hosted Checkout component in an Iframe.

The following sequence of steps need to be done in order to integrate Easypaisa Native Checkout with any e-commerce store/website.

Step 1: Input the required information

Easypaisa Native Checkout is a javascript function and in order to use this function, certain input parameters are required by the calling merchant website. Website owners must ensure that these input parameters are available inside website's code and are passed to the javascript function. The following inputs are needed from the website:

ID	Condition	Description
storeId	Mandatory	The store ID assigned to merchants by Easypaisa team.
amount	Mandatory	The transaction amount of the users' purchase (up to 6 digits and 2 decimal places).
orderId	Mandatory	The order ID assigned to the transaction.
email	Optional	Customers' email address.
cellNo	Optional	Customers' mobile number (11 digits starting with 03).
token	Mandatory	Expiry date and time for OTC token (Format: YYYYMMDD HHMMSS).
bankId	Optional	The Bank ID assigned to the bank. Can be used to restrict transactions from one bank.
encryptedHashRequest	Mandatory	Merchant will have to create this by encrypting request parameters against provided hash Key (Can be seen in merchant portal). Details are provided in section 4
Call Back URL	Optional	The Call back URL provided by merchant where the user must be redirected.
Signature	Optional	Signature of the merchant using the merchants private key
Payment Method	Optional	Merchant will pass an identifier to open a specific payment method of choice on the checkout page requested. Payment method tags: • OTC_PAYMENT_METHOD • MA_PAYMENT_METHOD • CC_PAYMENT_METHOD

ID	Condition	Description
		• QR_PAYMENT_METHOD • DD_PAYMENT_METHOD
timeStamp	Mandatory	Merchant will pass **current** datetime in below mentioned format: 2020-07-15T18:03:00

These input parameters can be collected through HTML input tags in an HTML form. The condition is that these input tags **must** have the "id" attribute, and value of this attribute must be those given in the table above.

The input tags can be of type "hidden" with values defined in the "value" attribute or can be entered by users through regular input fields.

Example

```
<! -- Store Id Provided by Easypay-->
<input name="storeId" id="storeId" value="43" hidden = "true"/>
<! -- Amount of Transaction from merchant's website -->
<input name="amount" id="amount" value="10" hidden = "true"/>
```

Step 2: Paste the Easypaisa script inside web-site

The following script must be inserted just before the end of the body tag of the website's HTML code.

Script:

```
<script>
        function getValues() {

        }
        function loadIframe(iframeName, url) {

           var storeID = document.getElementById("storeId").value;
           var amount = document.getElementById("amount").value;
           var orderID = document.getElementById("orderId").value;
           var email = document.getElementById("email").value;
           var cellNo = document.getElementById("cellNo").value;
           var token = document.getElementById("token").value;
           var bankId = document.getElementById("bankId").value;
           var postBackURL = document.getElementById("postBackURL").value;
           var merchantPaymentMethod  =
document.getElementById("merchantPaymentMethod").value;
```

```
            var signature  = document.getElementById("signature").value;

            var params = { storeId: storeID,  orderId: orderID, transactionAmount: amount,
mobileAccountNo: cellNo, emailAddress: email, transactionType: "InitialRequest", tokenExpiry: token,
bankIdentificationNumber: bankId, encryptedHashRequest:
encryptedHashRequest,merchantPaymentMethod : merchantPaymentMethod,
postBackURL:postBackURL,signature:signature};

            var $iframe = $('#' + iframeName);

            if ( $iframe.length ) {
               if(params.storeId != "" && params.orderId !="") {
                  var str = jQuery.param( params);
                  $iframe.attr('src',url+'?'+str);    // here you can change src
               }
            return false;
            }

            return true;
         }
         $( "#submitPaymentMethod" ).click(function() {
            $("#iframe-class").addClass("show-iframe");
         return loadIframe('easypay-iframe','http://localhost:3000/:0');

         });
      </script>
```

Step 3: Call the Easypaisa script

Integrating websites must contain a button, list, or any other html tag/element within the website, which can call the following javascript function on click:

loadIframe(iframeName, url).

Example:

```
<button type="submit" name="pay" class="btn btn-primary" id="submitPaymentMethod"
onClick="loadIframe();">Continue to Easypay Portal</button>
```

The example above creates a button in the website that loads the Easypaisa Native Checkout widget, on click.

Step 4: Create an iframe

Websites must also include an iframe in their payment page. This iframe will contain the Easypaisa checkout page. The following snippet can be used inside the merchant website, at the position where the iframe is needed:

```
<iframe id="easypay-iframe" name="easypay-iframe" src="about:blank" width="100%"
height="500px"></iframe>
```

Step 5: Sample String:

Before submitting request to payment gateway request, URL should be similar to the string below:

UAT:

https://easypaystg.easypaisa.com.pk/tpg/?storeId=1&orderId=6&transactionAmount=500.0&mobileAccountNo=03698965698&emailAddress=test@test.com&transactionType=InitialRequest&tokenExpiry=20201231%20111111&bankIdentificationNumber=&encryptedHashRequest=f4jnS1GpY97qR0KjjNutzdIbMhrtmgHObhMGWi9Sp7DS%2Bk%2FVM7usiV3K%2Fssyo8ULftZBUXkv%2FzduHaFcxq7dMRK6%2BgBppP61ZUQ%2BaOZzcXmKw6CgGCh71Vz3UuDulpDN

Query parameters can be a combination of any input parameters mentioned in step 1

PROD:

https://easypay.easypaisa.com.pk/tpg/?storeId=1&orderId=6&transactionAmount=500.0&mobileAccountNo=03698965698&emailAddress=test@test.com&transactionType=InitialRequest&tokenExpiry=20201231%20111111&bankIdentificationNumber=&encryptedHashRequest=f4jnS1GpY97qR0KjjNutzdIbMhrtmgHObhMGWi9Sp7DS%2Bk%2FVM7usiV3K%2Fssyo8ULftZBUXkv%2FzduHaFcxq7dMRK6%2BgBppP61ZUQ%2BaOZzcXmKw6CgGCh71Vz3UuDulpDN

Query parameters can be a combination of any input parameters mentioned in step 1

4. Encryption Algorithm

In order to mitigate parameter tempering/modification while transfer and posting of data, merchant will have to encrypt the request using the hash key provided in Merchant Portal. This encrypted request is sent along with the main request, which then reconciled at OPS end to detect if parameter is changed or not. The encryption can be done using following algorithm:

Step 1

Create a string in following format:

```
sampleString = amount=250&emailAddress=test@test.com&expiryDate= 20210101
151515&merchantPaymentMethod=MA_PAYMENT_METHOD&mobileNum=03698965698&orderRef
Num=hashTest&paymentMethod=InitialRequest&postBackURL=http://localhost/checkoutrevamp/&s
toreId=43&timeStamp=2020-07-15T18:03:00
```

If optional fields are not part of the request than they should also not be part of this string.

Step 2

Use AES/ECB/PKCS5Padding algorithm to encrypt with the key and string produced in the previous step

JAVA SAMPLE CODE:

```
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
SecretKeySpec   secretKey = new SecretKeySpec(key.getBytes(), "AES");
cipher.init(Cipher.ENCRYPT_MODE, secretKey);
encryptedValue = new String(Base64.encodeBase64(cipher.doFinal(sampleString.getBytes())));
```

PHP SAMPLE CODE:

```
$cipher = "aes-128-ecb";
$crypttext = openssl_encrypt($sampleString, $cipher, $hashKey,OPENSSL_RAW_DATA);
$hashRequest = base64_encode($crypttext);
```

JAVASCRIPT SAMPLE CODE:

```
var aesjs = require("aes-js")
const keyBuffer = aesjs.utils.utf8.toBytes(hashKey);
const inputBuffer = aesjs.padding.pkcs7.pad(aesjs.utils.utf8.toBytes(sampleString));
const escEcb = new aesjs.ModeOfOperation.ecb(keyBuffer);
const encryptedBytes = escEcb.encrypt(inputBuffer);
const encryptedData = Buffer.from(encryptedBytes).toString('base64');
```

5. Transaction Status on Postback URL

While redirecting to the merchant's postback URL the following information will be shared with the merchant, either at the time of Continue button click or the auto redirection after time is lapsed. This will provide visibility to the merchant while the user completes their transaction on the OPS checkout page.

The below information is shared with the merchant on the Postback URL in encoded format, as part of query string.

Field Name	Description
OTC	
paymentToken	OTC token as provided by IRIS
tokenExpiryDate	Token expiry time provided by IRIS
Amount	Amount of the transaction
orderRefNumber	Order reference number as provided by merchant
Status	Possible values: 0000: Success 0001: Failure
Description	If status parameter in post-back URL returns Error Code **0000 (Success)** then the user will be redirected to the OPS success page. In case of any other Error codes in post-back URL, failure will be considered.
MA	
orderRefNumber	Order reference number as provided by merchant
Amount	Amount of the transaction
TransactionRefNumber	Transaction ID provided by EWP in case of transaction success or failure
Status	Possible values: 0000: Success 0001: Failure
Description	If status parameter in post-back URL returns Error Code **0000 (Success)** then the user will be redirected to the OPS success page. In case of any other Error codes in post-back URL, failure will be considered.
CC – MIGS	
transactionResponse	Status of transaction as per payment gateway
amount	Amount of the transaction
orderRefNumber	Order reference number as provided by merchant
TransactionNumber	Transaction ID provided by payment gateway
AuthorizeId	Auth ID of the transaction
BatchNumber	Batch number of the CC transaction

Status	Possible values: 0000: Success 0001: Failure
Description	If status parameter in post-back URL returns Error Code **0000 (Success)** then the user will be redirected to the OPS success page. In case of any other Error codes in post-back URL, failure will be considered.
CC – MPGS	
TransactionResponse	Status of transaction as per payment gateway
amount	Amount of the transaction
orderRefNumber	Order reference number as provided by merchant
TransactionRefNumber	Transaction ID provided by payment gateway
AuthorizeId	Auth ID of the transaction
BatchNumber	Batch number of the CC transaction
Status	Possible values: 0000: Success 0001: Failure
Description	If status parameter in post-back URL returns Error Code **0000 (Success)** then the user will be redirected to the OPS success page. In case of any other Error codes in post-back URL, failure will be considered.
DD	
orderRefNumber	Order reference number as provided by merchant
Amount	Amount of the transaction
TransactionRefNumber	Transaction ID provided by Apps in case of transaction success or failure
Status	Possible values: 0000: Success 0001: Failure
Description	If status parameter in post-back URL returns Error Code **0000 (Success)** then the user will be redirected to the OPS success page. In case of any other Error codes in post-back URL, failure will be considered.

6. RSA Encryption (Optional Pre Requisite)

In order to use the RSA encryption feature of hosted checkout, client have to send signature in the request. This signature is used by Easypay to authenticate the client before processing the request. The steps required to perform this encryption are described below:

Client and Easypaisa must exchange RSA keys before passing signature in input, and the length of RSA key must be 2048 bits. When making call to Easypaisa, client uses the RSA private key to sign the API request. After receiving the API request, Easypay will use the client's RSA public key to verify whether the signature is matched to the content of API request.

Public/Private Key Exchange

An RSA key pair contains the private key and the public key. The private key is required for generating the signature. Many tools can be used to generate the RSA key pair. After the RSA2 key pair is generated, you must exchange the public key with the Easypay server for signature verification by completing the following steps:

1. Upload your public key in Easypay merchant portal.

Log in to the portal and visit the following path: "*Account Settings -> Public Key Configuration -> Upload Public Key*". Upload your public key here.

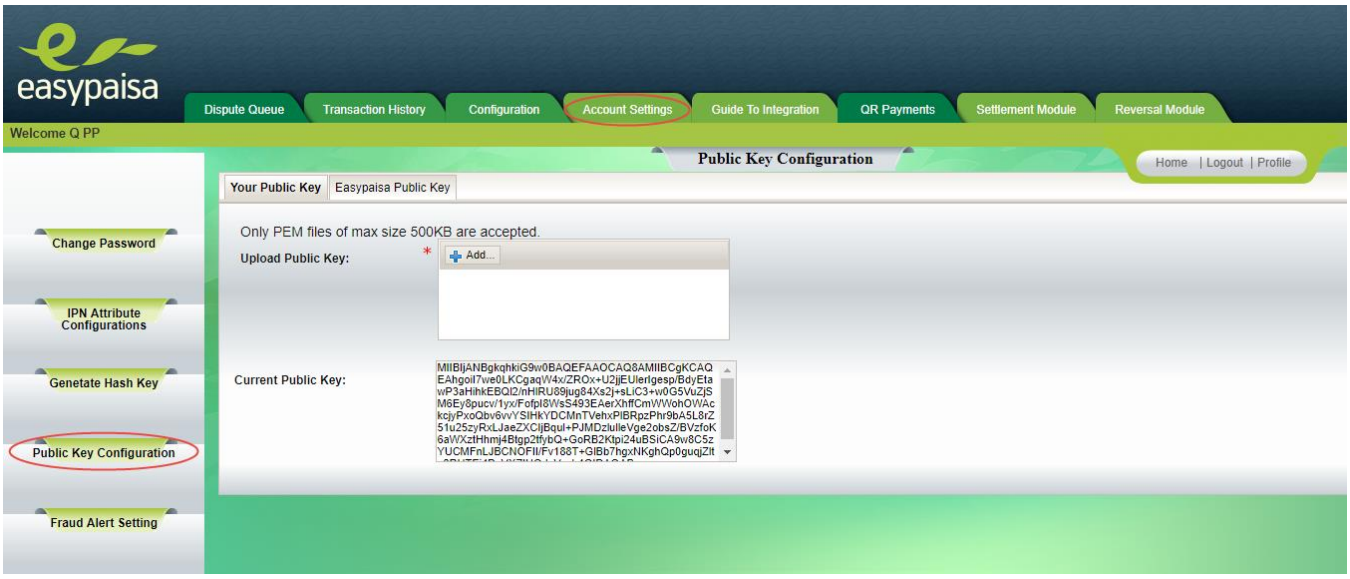


Figure 2 – Public Key Configuration (Upload)

Signature Generation

Generate signature using the following steps:

1. Extract the content of the request to be signed in JSON format and trim un-necessary spaces so that it looks like this:

{"storeId":"43","transactionAmount":"100","orderId":"Test-new-002","mobileAccountNo":"03656565667","emailAddress":"Test@gmail.com","transactionType":"InitialRequest","bankIdentificationNumber":"","tokenExpiry":"","merchantPaymentMethod":"","postBackURL":"http://sl-tpg-57674.systemsltd.local:9081/TestStore"}

Note: The following sequence of parameters must be followed to generate the signature:

{"storeId":"43","transactionAmount":"100","orderId":"Test-new-002","mobileAccountNo":"03656565667","emailAddress":"Test@gmail.com","transactionType":"InitialRequest","bankIdentificationNumber":"","tokenExpiry":"","merchantPaymentMethod":"","postBackURL":"http://sl-tpg-57674.systemsltd.local:9081/TestStore"}

2. Hash the above mentioned JSON content by using the SHA256withRSA algorithm. Then use the client's RSA private key to sign the value to obtain the signature. To achieve a better security level, the length of the RSA key/pairs must be 2048 bits. The following sample illustrates a raw signature:

sXSGdA4tR4B0wnF6Nr8ShpfNMtHISWuzLpdKoNWskIgCcspMVFFzgkez4BrmtYXkMqZh1tyz/8sNNU3F
2VTu0fOx+ZP4CnmVjNu8bciu4hTtnytA3Y1GZ/yXAUD+mVYGXeyDO3fHrqDdI7k3xV/Q8FCOd0culTsN
6RRMxMZq+oqR2x+EjOXVofp7BZJwhQE8Us4Al573FQz5EHDvCLx35lq2hoMhXLCVLVDBHcpCgotuY4mg
91MXh/qqcFGY/hQKhGdWAbZ9tcp2Q5s2ESh37rqxnOwZrrepK8NC6vMBPUiSPxQefpev2nG0JzAHOjRP
bBXYw1XBS6RBYKAmufZZiQ==

3. Base64-encode the signature. A sample output is given below:

c1hTR2RBNHRSNEIwd25GNk5yOFNocGZOTXRISVNXdXpMcGRLb05Xc2tJZ0Njc3BNVkZGemdrZXo0QnJt
dFlYa01xWmgxdHl6LzhzTk5VM0YyVlR1MGZPeCtaUDRDbm1Wak51OGJjaXU0aFR0bnl0QTNZMUdaL3lY
QVVEK21WWUdYZXlETzNmSHJxRGRJN2szeFYvUThGQ09kMGN1bFRzTjZSUk14TVpxK29xUjJ4K0VqT1hW
b2ZwN0JaSndoUUU4VXM0QWw1NzNGUXo1RUhEdkNMeDM1bHEyaG9NaFhMQ1ZMVkRCSGNwQ2dvdHVZNG1n
OTFNWGgvcXFjRkdZL2hRS2hHZFdBYlo5dGNwMlE1czJFU2gzN3JxeG5Pd1pycmVwSzhOQzZ2TUJQVWlT
UHhRZWZwZXYybkcwSnpBSE9qUlBiQlhZdzFYQlM2UkJZS0FtdWZaWmlRPT0=

4. Use the obtained string as the value of the Signature parameter like below:

https://easypaystg.easypaisa.com.pk/tpg?storeId=43&orderId=123456&transactionAmount=100&mobileAccountNo=03215201892&emailAddress=beenishsajjad277%40gmail.com&transactionType=InitialRequest&tokenExpiry=&bankIdentificationNumber=&hashkey=1I1I9G9540UGXHIK&merchantPaymentMethod=&postBackURL=&signature=NuFCsLR1dM%2Fs68gkLpyPCfQYesu8KyCYHRMT%2FLgoZcsUTJD1EqY03DbOXdTcP77E7PTJgPbS8K0qhxDpxJ8vSBl9FTxrReIl%2F%2FfTKsd1RkwygBM2Gdj26ilEjfFAh%2BRVX3hZgMlqOId0xNk1DwrJvlRLvOP2Fd9MZG5KH5BP5BA%2BPZXWxinSxWbKIXu1LZikYmBdGc5beawBr2F5yx%2BTbGO3GH4EKlA99kTjuR%2F%2Bii3InED8kcrYp63U2MWlAgBp5rWqZpRvcWzi84CHQzm%2F3gPcWexEia5iZr9OYFlszhZUywQZGkI4fz14JshCNVztEf4f88TJSTYhoCl2FUegDA%3D%3D

Note: Signature inclusion in request is optional.

7. IPN Integration

Instant payment notification message is used to notify merchants about the details of any particular transaction made by customers using payment gateway channel. IPN message is self-configured in Payment Gateway for each merchant. It is customized as to what details merchant requires in response for any particular transaction. The IPN message service sends merchant a notification whenever Payment Gateway transaction is created (with 'Paid' status) or is updated to 'Paid'.

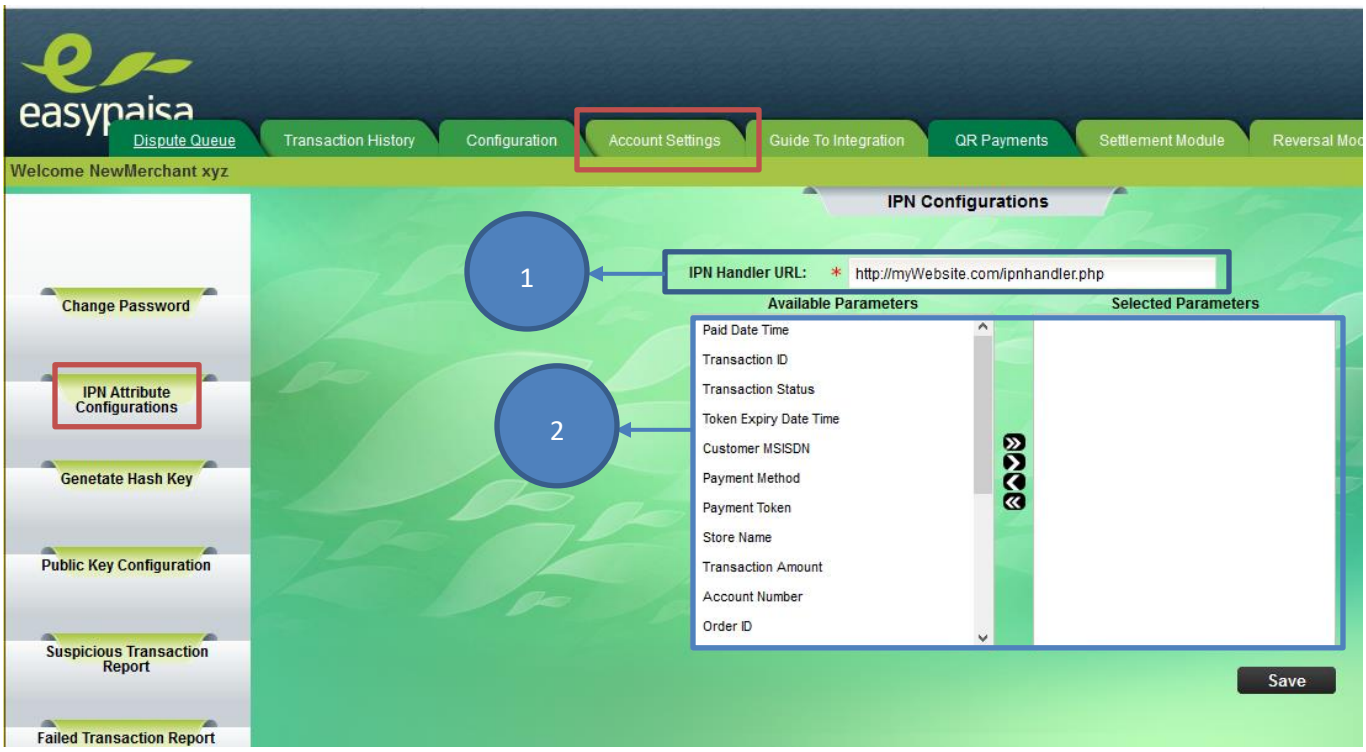
IPN Handler URL is the Merchant's IPN listener URL where Merchant will be expecting the response from Payment Gateway. IPN message will contain the selected parameters configured for merchant using below mentioned screen in merchant portal.

Payments Flow

1. Transaction is initiated by the customer through merchant's site on check out page.
2. Easypay will send an IPN REST URL to the IPN listener that is configured by the merchant via Easypay merchant portal
3. Merchant will then hit the received IPN URL and fetch the configured attribute.

How to Integrate IPN Service

1. Configure the listener URL on Easypay merchant portal.
2. Configure the desired IPN attributes from the Easypay merchant portal
3. Create a listener that will receive in GET request variable named 'url'.
4. The received URL would be that of REST API which will return the IPN attributes.



Following is the format for URL:

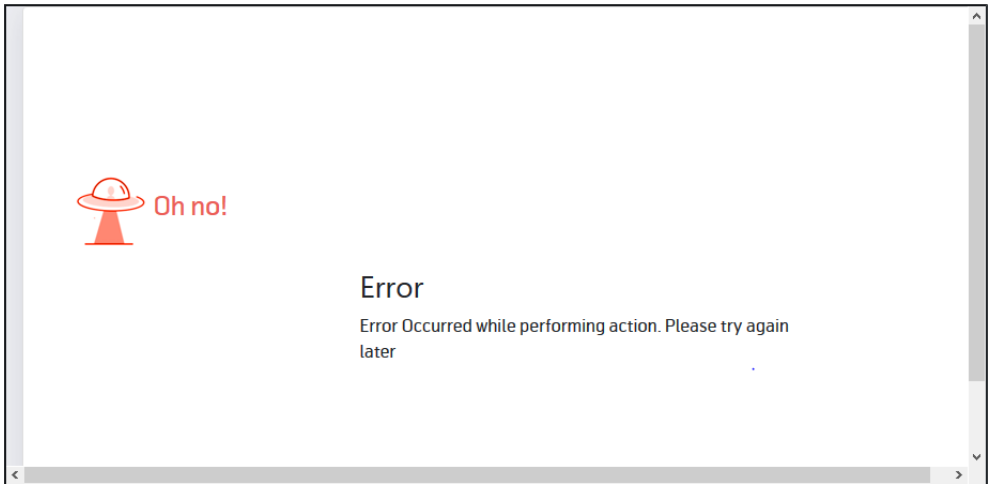
MerchantURL?url=Rest API URL/Merchant Account ID/Order ID Where url = is the reserved word added by the system.

For Example: Merchant URL = http://www.TestMerchant.com Rest API URL = https://easypay.easypaisa.com.pk/easypay-service/rest/v1/order- status Merchant Account ID = 00001 OrderID = 998877 IPN Message: https://www.TestMerchant.com?url= https://easypay.easypaisa.com.pk/easypay-service/rest/v1/order- status/1000223344/998877

8. Errors in Payment Methods

If the request sent by merchant contains invalid data like incorrect store id, invalid amount, missing mandatory parameter then following error will be shown on screen.

In this case client needs to check their request parameters:



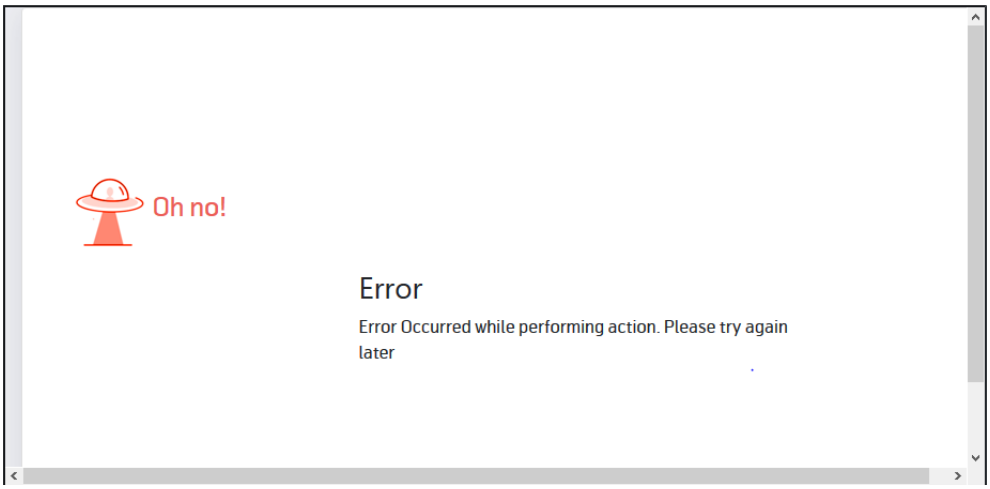
MA

In case of any error in transaction, following error will be displayed:

"Sorry, your payment has failed. Please try again"

OTC

In case of any error in transaction, following error will be displayed:



The error depicts a problem in transaction request and it must be fixed to proceed without an error.

DD

In case of any error in DD transactions, following generic error will be visible on screen"

> *"Some problem occured in your transaction. Please contact system administrator for further assistance."*

CC

1. In case of any configurational issue, any of the following error will appear on screen. The merchant must contact Telenor Bank resource in case of any of the following errors:
    a. *"Some problem occurred in your transaction. Please contact system administrator for further assistance."*
    b. *"Invalid card number"*
    c. *"This is not a valid card number"*
    d. *"We regret to inform you that payment with credit card issued by your respective bank is not allowed. Kindly use another credit card or payment method. Thank you."*
    e. *"BIN not properly defined for routing"*
2. In case of any transactional issue, any of the following error will appear on screen. The merchant must contact Telenor Bank resource in case of any of the following errors:
    a. *"Your card seems to be expired, please recheck your card's expiry date."*
    b. *"Transaction time out"*
    c. *"Declined by Card Issuer"*
    d. *"Please contact your card issuing bank or use another card"*
    e. *"Your transaction is not authorized. Please contact your card issuing bank or use another card"*
    f. *"No response from the card issuing bank. Please contact your bank or use an another card"*